

NAME : GALLA MOHAN SAI

1) Spring Data JPA - Employee Management Example

application.properties

Application Logs

logging.level.org.springframework=info

logging.level.com.employee=debug

Hibernate SQL Logs

logging.level.org.hibernate.SQL=trace

logging.level.org.hibernate.type.descriptor.sql=trace

Database Configuration

spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.datasource.url=jdbc:mysql://localhost:3306/employeeedb

spring.datasource.username=root

spring.datasource.password=root

Hibernate Configuration

spring.jpa.hibernate.ddl-auto=validate

spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect

MySQL Table

CREATE DATABASE employeeedb;

USE employeeedb;

```
CREATE TABLE employee (  
    id INT PRIMARY KEY,  
    name VARCHAR(50),  
    department VARCHAR(50)  
);
```

INSERT INTO employee VALUES (101, 'Rahul', 'CSE');

INSERT INTO employee VALUES (102, 'Anjali', 'ECE');

Employee.java

```
package com.employee.model;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name="employee")
public class Employee {

    @Id
    @Column(name="id")
    private int id;

    @Column(name="name")
    private String name;

    @Column(name="department")
    private String department;

    public Employee() {
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getDepartment() {
```

```

        return department;
    }

    public void setDepartment(String department) {
        this.department = department;
    }

    @Override
    public String toString() {
        return "Employee [id=" + id +
            ", name=" + name +
            ", department=" + department + "]";
    }
}

```

EmployeeRepository.java

```

package com.employee.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import com.employee.model.Employee;

@Repository
public interface EmployeeRepository
    extends JpaRepository<Employee, Integer> {

}

```

EmployeeService.java

```

package com.employee.service;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

```

```
import com.employee.model.Employee;
import com.employee.repository.EmployeeRepository;
```

```
@Service
```

```
public class EmployeeService {
```

```
    @Autowired
```

```
    private EmployeeRepository employeeRepository;
```

```
    @Transactional
```

```
    public List<Employee> getAllEmployees() {
```

```
        return employeeRepository.findAll();
```

```
    }
```

```
}
```

EmployeeApplication.java

```
package com.employee;
```

```
import java.util.List;
```

```
import org.slf4j.Logger;
```

```
import org.slf4j.LoggerFactory;
```

```
import org.springframework.boot.SpringApplication;
```

```
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
import org.springframework.context.ApplicationContext;
```

```
import com.employee.model.Employee;
```

```
import com.employee.service.EmployeeService;
```

```
@SpringBootApplication
```

```
public class EmployeeApplication {
```

```
    private static final Logger LOGGER =
```

```
        LoggerFactory.getLogger(EmployeeApplication.class);
```

```
private static EmployeeService employeeService;

public static void main(String[] args) {

    ApplicationContext context =
        SpringApplication.run(EmployeeApplication.class, args);

    employeeService = context.getBean(EmployeeService.class);

    LOGGER.info("Application Started");

    displayEmployees();

}

private static void displayEmployees() {

    LOGGER.info("Fetching Employee Records");

    List<Employee> employees =
        employeeService.getAllEmployees();

    LOGGER.debug("Employees = {}", employees);

    LOGGER.info("Completed");

}

}
```

Expected Output

Application Started

Fetching Employee Records

Employees =

Employee [id=101, name=Rahul, department=CSE]

Employee [id=102, name=Anjali, department=ECE]

Completed

2) Difference Between JPA, Hibernate and Spring Data JPA

Feature	JPA	Hibernate	Spring Data JPA
Type	Specification	ORM Framework	Spring Framework
Developed By	Jakarta EE	Hibernate Team	Spring
Implementation	No	Yes	Uses JPA
SQL Generation	No	Yes	Yes
Boilerplate Code	More	Less	Very Less
CRUD Support	Manual	Available	Automatic
Repository Support	No	No	Yes (JpaRepository)

JDBC(Without hybernity)

```
Connection con = DriverManager.getConnection(url, user, password);
```

```
PreparedStatement ps =  
con.prepareStatement("SELECT * FROM employee");
```

```
ResultSet rs = ps.executeQuery();
```

Hibernate

```
Session session = sessionFactory.openSession();
```

```
Transaction tx = session.beginTransaction();
```

```
session.save(employee);
```

```
tx.commit();
```

```
session.close();
```

Spring Data JPA

Repository

```
public interface EmployeeRepository  
extends JpaRepository<Employee, Integer> {  
  
}
```

Service

```
@Autowired  
private EmployeeRepository employeeRepository;  
  
@Transactional  
public void addEmployee(Employee employee) {  
  
    employeeRepository.save(employee);  
  
}
```

Flow of Execution

EmployeeApplication

↓

EmployeeService

↓

EmployeeRepository

↓

Spring Data JPA

↓

Hibernate



MySQL Database



Employee Records Returned